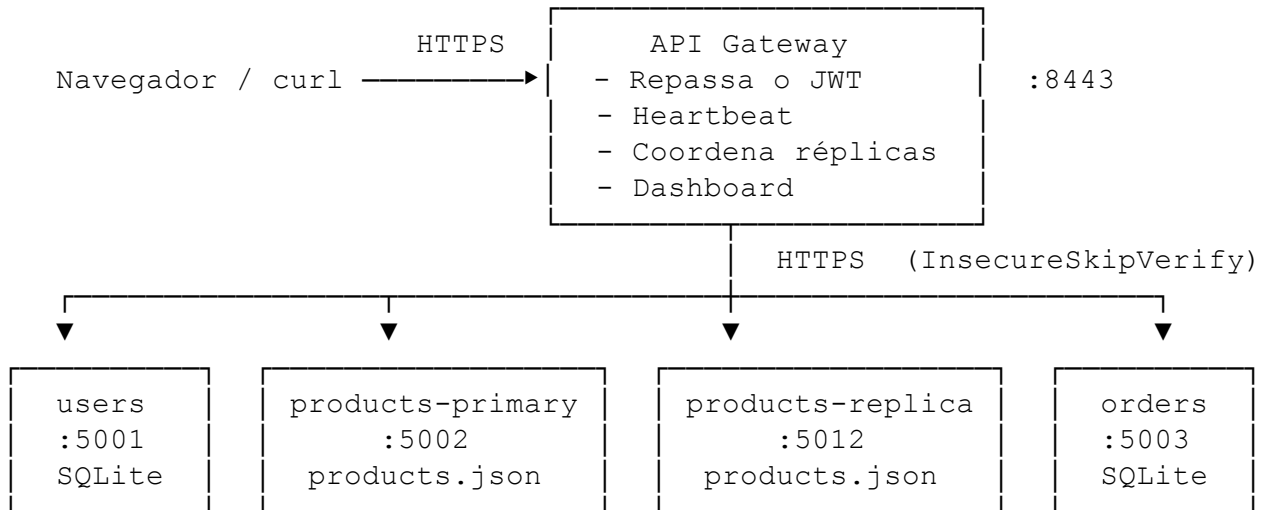


Relatório - Mini E-Commerce Distribuído

Arquitetura

O sistema é composto por cinco processos rodando ao mesmo tempo, que se comunicam via HTTPS:



Todos os serviços podem ser verificados com `/health` e `/admin/toggle`. Para testar a detecção se um serviço for derrubado, foram adicionados botões na dashboard para derrubar os serviços. O restart funciona assim:

1. A diretriz “restart: unless-stopped” do Compose levanta o container de novo automaticamente.
2. O heartbeat do gateway detecta a queda rápida, marca o serviço como DOWN, e registra RECOVERED no primeiro contato bem-sucedido depois que o container volta.

1 Como os serviços se comunicam?

REST/JSON por meio de HTTPS. Não foi implementada fila de mensagens. O fluxo é:

1. O cliente manda uma requisição HTTPS para o api gateway na porta 8443.
2. O gateway verifica qual serviço é o destino pelo prefixo da URL e usa o `http.Client` interno pra fazer uma requisição HTTPS para o serviço apropriado.
3. O serviço de destino valida o JWT com o `JWT_SECRET` compartilhado e processa a requisição.
4. A resposta volta pelo gateway até o cliente.

Internamente o gateway usa `tls.InsecureSkipVerify=true` porque o certificado auto-assinado é compartilhado entre todos os containers.

Em produção seria utilizada uma cadeia assinada por uma CA (e preferencialmente mTLS).

2 Qual estratégia de consistência foi adotada?

Consistência forte no catálogo de produtos. O ProductReplicaManager do gateway dispara as escritas (POST /products) para as duas réplicas em paralelo via duas goroutines, espera as duas respostas, e só devolve sucesso quando as duas respondem 2xx. As leituras fazem round-robin entre as réplicas e se a réplica escolhida estiver marcada como indisponível pelo heartbeat, a requisição cai automaticamente na outra.

O catálogo é lido muito mais do que é escrito, e algo desatualizado (como "o produto que acabei de criar não aparece") poderia ser confuso nesse app aqui. Como só tem duas réplicas, o custo da consistência forte é uma queda negligível na disponibilidade de escrita. Se uma das duas estiver fora, a escrita falha e retorna o status 500, o que é uma escolha consciente:

Preocupação	Decisão	Custo
Leitura	Forte	Acoplamento maior na escrita
Disponibilidade de escrita	Reduzida	Escrita bloqueada quando 1 das 2 réplicas cai
Disponibilidade de leitura	Aumentada	Leitura funciona se qualquer uma das duas estiver no ar

Num sistema real de produção, para um catálogo real com muitas réplicas, a consistência passaria a ser eventual (replicação assíncrona ou escrita por quórum) porque a probabilidade de ter todas as réplicas no ar o tempo todo pode cair rápido conforme o número delas cresce.

No caso de falha parcial, (e.g. a réplica A aceita e a B falha), o gateway devolve 500 e loga a inconsistência. Não foi adicionado rollback automático. Não foi implementado two-phase commit ou um log replicado idempotente. Um sistema real usaria 2PC ou um log replicado (Raft, Kafka).

3 O que acontece se o serviço de Pedidos cair?

O heartbeat do gateway manda um GET /health pra cada serviço a cada 5 segundos. Depois de duas falhas consecutivas (FailuresBeforeMarkingDown = 2), o registry vira o serviço de pedidos pra DOWN, registra o evento, e joga ele pro buffer de eventos do dashboard.

A partir daí:

- Qualquer requisição em /api/orders/ devolve HTTP 503 imediatamente, sem nem encostar na rede. O ProxyClient.HandlerFor do gateway para quando availabilityRegistry.IsAvailable("orders") é falso.
- O serviço de usuários e o de produtos continuam funcionando normalmente (login, cadastro, navegar no catálogo e criar produto). As falhas ficam isoladas.
- Pedidos feitos durante a queda são perdidos enquanto o serviço estiver fora. Não tem fila de retry. Um sistema real usaria um sistema de mensagem como Kafka ou RabbitMQ, ou aceitaria as requisições de forma otimista pra reconciliar depois.

Quando o serviço volta, o próximo probe bem-sucedido vira ele pra UP de novo, registra um evento RECOVERED, e o tráfego volta imediatamente. O ciclo completo (cair, detectar, reiniciar, recuperar) está implementado.

O botão Kill do dashboard simula uma queda com um shutdown. O restart do Compose sobe o container de novo, e o caminho de recuperação é exercitado de ponta a ponta sem intervenção manual.

4 Como o JWT impede um usuário comum de criar produtos?

O POST /products é protegido por dois middlewares aplicados como sub-grupo do chi:

```
router.Group(func(administratorRouter chi.Router) {  
  
    administratorRouter.Use(authentication.RequireValidToken(signingSecret))  
    administratorRouter.Use(authentication.RequireAdministratorRole)  
    administratorRouter.Post("/products", writeCreateHandler(productStore))  
})
```

O primeiro middleware faz três coisas:

1. Pega o token Bearer do header Authorization.
2. Valida a assinatura HMAC-SHA256 com o JWT_SECRET. A biblioteca está fixada nesse algoritmo, então não dá pra trocar o alg pra none.
3. Confere o claim exp e rejeita tokens expirados.

Se qualquer uma dessas falhar, a requisição volta com 401.

O segundo middleware lê o claim role do payload e rejeita qualquer coisa que não seja admin com 403. Como o JWT é assinado, o usuário não consegue alterar o claim role (a assinatura não bateria mais) nem gerar um token novo dizendo role=admin (não tem a chave secreta).

A rota de login reforça isso: o serviço de usuários consulta a coluna role no SQLite (default 'user' no cadastro) e coloca esse valor no token emitido. O único jeito de conseguir um token de admin é fazer login em uma conta admin que já existe.

5 Limitações em relação a um sistema de produção real

- **Chave única de JWT compartilhada.** Todos os serviços usam a mesma chave HMAC. Sistemas reais usam assinatura assimétrica (RS256/ES256), onde só o serviço de auth tem a chave privada e os outros validam com a pública.

- **InsecureSkipVerify no HTTPS interno.** Os certificados auto-assinados estão dentro de cada imagem e a verificação está desligada. Em produção precisaria de uma cadeia assinada por uma CA, idealmente com mTLS.

- **Sem resolução de conflito entre as réplicas de produtos.** As duas aceitam escritas idênticas pelo gateway, mas se uma partição de rede deixasse elas divergirem, não tem lógica de merge. O contorno é rotear todas as escritas pelo gateway e devolver erro em falha parcial.

- **Sem rollback automático em falha parcial de réplica.** Se a réplica A aceita e a B falha, o gateway loga e devolve 500. Não tenta desfazer a escrita que deu certo.

- **Sem service discovery.** As URLs dos serviços são passadas como variável de ambiente. Em produção seria Consul, DNS do Kubernetes, ou um service mesh.
- **Sem rate limiting nem observabilidade.** Não tem métrica Prometheus, trace OpenTelemetry, nem coleta estruturada de log de acesso.
- **Sem transação entre serviços.** Um pedido pode referenciar um produto deletado, porque o serviço de pedidos não consulta o de produtos. Foi de propósito pra deixar o modelo de falha mais simples, mas um sistema real validaria via log de eventos ou aceitaria a inconsistência eventual.
- **Estado do kill switch e buffer de eventos em memória.** Os dois se perdem quando o container reinicia. OK pro demo, inútil como trilha de auditoria.
- **Gateway com instância única.** Um deploy real teria várias instâncias atrás de um balanceador. Aqui é ponto único de falha.
- **Sem proteção no /admin/toggle.** Quem tem acesso de rede ao serviço pode derrubar ele. Em produção ficaria atrás de um endpoint admin autenticado.

6 Onde achar cada coisa

Requisito	Onde está
Roteamento do gateway + repasse de JWT	internal/gateway/proxy.go, internal/gateway/server.go
Serviço de usuários (cadastro, login, get)	internal/users/handlers.go, internal/users/store.go
Serviço de produtos (listar, detalhar, criar)	internal/products/handlers.go, internal/products/store.go
Serviço de pedidos (criar, listar)	internal/orders/handlers.go, internal/orders/store.go
JWT (HS256, exp, role)	internal/authentication/authentication.go
Hash de senha (bcrypt)	HashPassword / VerifyPassword
Proteção de admin	middleware RequireAdministratorRole
Duas réplicas de produto	products-primary, products-replica no docker-compose.yml
Replicação com consistência forte	internal/gateway/replica.go (HandleWrite)
Leitura round-robin	internal/gateway/replica.go (HandleRead)
Heartbeat (5 s, 2 falhas)	internal/gateway/heartbeat.go
Log de falha e recuperação	slog.Info("service recovered"), slog.Warn("service marked down")
/health em todos os serviços	BuildRouter de cada serviço
Dashboard	internal/gateway/web/dashboard.html, internal/gateway/dashboard.go
Docker Compose (bônus)	Dockerfile, docker-compose.yml
HTTPS/TLS (bônus)	internal/tlsserver/tlsserver.go, certs/generate.sh
Dashboard de monitoramento (bônus)	servido em https://localhost:8443/dashboard